

16 Working with Spring Data REST

This chapter covers

- Introducing REST applications
- Creating a Spring Data REST application
- Using ETags for conditional requests
- Limiting access to repositories, methods, and fields
- Working with REST events
- Using projections and excerpts

Representational state transfer (REST) is a software architectural style for creating web services; it also provides a set of constraints. The American computer scientist Roy Fielding, who is also one of the authors of the HTTP specification, first defined REST, presenting the REST principles in his PhD dissertation (Fielding, 2000). Web services following this REST architectural style are called *RESTful web services*, and they allow interoperability between the internet and computer systems. Requesting systems can access and manipulate web resources represented as text using a well-known set of stateless operations (GET , POST , PUT , PATCH , DELETE). A stateless operation does not depend on any other prior operation; it must contain all the information needed to be understood by the server.

16.1 Introducing REST applications

We will first define the terms *client* and *resource* to describe what makes an API RESTful. A *client* is a person or software using the RESTful API. For example, a programmer using a RESTful API to execute actions against the LinkedIn website is a client, but the client can also be a web browser. When we go to the LinkedIn website, our browser is the client that calls the website API and that displays the obtained information on the screen. A *resource* can be any object about which the API can obtain information. In the LinkedIn API, a resource can be a message, a photo, or a user. Each resource has a unique identifier.

The REST architecture style defines six constraints

(<https://restfulapi.net/rest-architectural-constraints/>):

- *Client-server*—Clients are separated from servers, and each has its own concerns. Most frequently, a client is concerned with the representation of the user, and a server is concerned with data storage and domain model logic—the conceptual model of a domain including data and behavior.
- *Stateless*—The server does not keep any information about the client between requests. Each request from a client contains all of the information necessary to respond to that request. The client keeps the state on its side.
- *Uniform interface*—The client and the server may evolve independently of each other. The uniform interface between them makes them loosely coupled.
- *Layered systems*—The client does not have any way to determine if it is interacting directly with the server or with an intermediary. Layers can be dynamically added and removed. They can provide security, load balancing, or shared caching.
- *Cacheable*—Clients are able to cache responses. Responses define themselves as cacheable or not.
- *Code on demand (optional)*—Servers are able to temporarily customize or extend the functionality of a client. The server can transfer some logic to the client that the client can execute, such as JavaScript client-side scripts.

A RESTful web application provides information about its resources, which are identified with the help of URLs. The client can execute actions against such a resource; it can create, read, update, or delete a resource.

The REST architectural style is not protocol-specific, but the most widely used protocol is REST over HTTP. HTTP is a synchronous application network protocol based on requests and responses.

To make our API RESTful, we have to follow a set of rules while developing it. A RESTful API will transfer information to the client, which uses that information as a representation of the state of the accessed resource. For example, when we call the LinkedIn API to access a specific user, the API will return the state of that user (name, biography, professional experience, posts). The REST rules make the API easier to understand and simpler for new programmers to use when they join a team.

The representation of the state can be in JSON, XML, or HTML format. The client uses the API to send the following information to the server:

- The identifier (URL) of the resource we want to access.
- The operation we want the server to perform on that resource. This is an HTTP method, the most common of which are `GET`, `POST`, `PUT`, `PATCH`, and `DELETE`.

For example, using the LinkedIn RESTful API to fetch a specific LinkedIn user requires that we have a URL that identifies the user and that we use the HTTP method `GET`.

16.2 Creating a Spring Data REST application

Our first goal is to create a Spring Data REST application that will provide a browser interface to interact with the database and to manage and persist CaveatEmptor users. To do this, we'll access the Spring Initializr website (<https://start.spring.io/>) and create a new Spring Boot project (figure 16.1), having the following characteristics:

- Group: `com.manning.javapersistence`
- Artifact: `spring-data-rest`
- Description: Spring Data REST

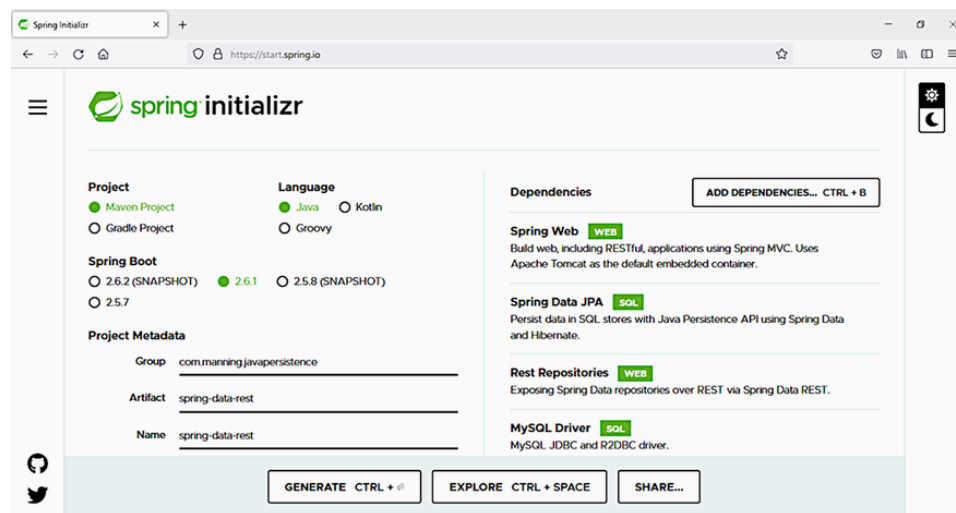


Figure 16.1 Creating a new Spring Boot project using Spring Data REST and MySQL

We'll also add the following dependencies:

- Spring Web (this will add `spring-boot-starter-web` in the Maven `pom.xml` file)
- Spring Data JPA (this will add `spring-boot-starter-data-jpa` in the Maven `pom.xml` file)
- REST Repositories (this will add `spring-boot-starter-data-rest` in the Maven `pom.xml` file)
- MySQL Driver (this will add `mysql-connector-java` in the Maven `pom.xml` file)

NOTE To execute the examples from the source code, you'll first need to run the `Ch16.sql` script.

The `pom.xml` file in the following listing includes the dependencies we added to start the Spring Data REST project. This Spring Data REST application will access a MySQL database, so we need the driver.

Path: Ch16/spring-data-rest/pom.xml

```

<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-rest</artifactId>
</dependency>
<dependency>
    <groupId>mysql</groupId>
    <artifactId>mysql-connector-java</artifactId>
    <scope>runtime</scope>
</dependency>

```

Ⓐ
Ⓐ
Ⓐ
Ⓐ
Ⓑ
Ⓑ
Ⓑ
Ⓑ
Ⓒ
Ⓒ
Ⓒ
Ⓒ
Ⓓ
Ⓓ
Ⓓ
Ⓓ

Ⓐ `spring-boot-starter-web` is the starter dependency used by Spring Boot to build web applications.

Ⓑ `spring-boot-starter-data-jpa` is the starter dependency used by Spring Boot to connect to a relational database through Spring Data JPA.

Ⓒ `spring-boot-starter-data-rest` is the starter dependency used by Spring Boot for Spring Data REST applications.

Ⓓ `mysql-connector-java` is the JDBC driver for MySQL. It is a runtime dependency, so it is needed in the classpath only at runtime.

The next step is to fill in the Spring Boot `application.properties` file, which can include various properties to be used by the application. Spring Boot will automatically find and load the `application.properties` file from the classpath—the `src/main/resources` folder is added by Maven to the classpath.

There are several ways to provide parameters in a Spring Boot application, and the `.properties` file is just one of them. Parameters can also come from the source code or as command-line arguments—see the Spring Boot documentation for details.

For our application, the `application.properties` configuration file will look like the following listing.

```
Path: Ch16/spring-data-rest/src/main/resources/application.properties

server.port=8081
spring.datasource.url=jdbc:mysql://localhost:3306/CH16_SPRINGDATAREST
➡ ?serverTimezone=UTC
spring.datasource.username=root
spring.datasource.password=
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dia
spring.jpa.show-sql=true
spring.jpa.hibernate.ddl-auto=create
```

- Ⓐ The application will start on port 8081.
- Ⓑ The URL of the database.
- Ⓒ The credentials to access the database. Replace them with the credentials on your machine, and use a password in real life.
- Ⓓ The dialect of the database, MySQL.
- Ⓔ Show the SQL queries when they are executed.
- Ⓕ Recreate the tables for each execution of the application.

The `User` class will now contain a `@Version` annotated field. As discussed in section 11.2.2, the field value is incremented whenever a modified `User` instance has been persisted. Section 16.3 will demonstrate how this field can be used for conditional REST requests using ETags.

Listing 16.3 The modified `User` class

```
Path: Ch16/spring-data-rest/src/main/java/com/manning/javapersistence/c
➡ /model/User.java

@Entity
public class User {

    @Id
    @GeneratedValue
    private Long id;

    @Version
    private Long version;

    private String name;
    private boolean isRegistered;

    private boolean isCitizen;
```

```

        //constructors, getters and setters
    }

```

Users will participate in an auction that is represented by the `Auction` class. An auction is described by the `auctionNumber`, the number of `seats`, and the set of `users`.

Listing 16.4 The `Auction` class

```

Path: Ch16/ spring-data-rest/src/main/java/com/manning/javapersistence/
    ➡ /model/Auction.java

public class Auction {

    private String auctionNumber;
    private int seats;
    private Set<User> users = new HashSet<>();

    //constructors, getters and methods
}

```

The users participating in the auction will be read from a CSV file, in the `CsvDataLoader` class. We'll use the `@Bean` annotation to create a bean that will be managed and injected into the application by Spring.

Listing 16.5 The `CsvDataLoader` class

```

Path: Ch16/spring-data-rest/src/main/java/com/manning/javapersistence/c
    ➡ /beans/CsvDataLoader.java

public class CsvDataLoader {

    @Bean
    public Auction buildAuctionFromCsv() throws IOException {
        Auction auction = new Auction("1234", 20);
        try (BufferedReader reader = new BufferedReader(
            new FileReader("src/main/resources/users_information.csv"))
            String line = null;
            do {
                line = reader.readLine();
                if (line != null) {
                    User user = new User(line);
                    user.setIsRegistered(false);
                    auction.addUser(user);
                }
            } while (line != null);

        }

        return auction;
    }
}

```

```
}  
}
```

- Ⓐ The result of the method will be a bean managed by Spring.
- Ⓑ Create the `Auction` object.
- Ⓒ Use the information from the CSV file.
- Ⓓ Read line by line.
- Ⓔ Create the user from the read information, configure it, and add it to the auction.
- Ⓕ Return the `Auction` bean.

The `UserRepository` interface extends `JpaRepository<User, Long>`, inheriting the JPA-related methods and managing the `User` entity, having IDs of type `Long`.

Listing 16.6 The `UserRepository` interface

```
Path: Ch16/spring-data-rest/src/main/java/com/manning/javapersistence/c  
➡ /repositories/UserRepository.java  
  
public interface UserRepository extends JpaRepository<User, Long> {  
  
}
```

The Spring Boot application will import the bean created in the `CsvDataLoader` class and autowire it. It will also create a bean of type `ApplicationRunner`. This is a Spring Boot functional interface (an interface with a single abstract method) that gives access to application arguments. This `ApplicationRunner` interface is created, and its single method is executed, just before the `run()` method from `SpringApplication` completes.

Listing 16.7 The `Application` class

```
Path: Ch16/spring-data-rest/src/main/java/com/manning/javapersistence/c  
➡ /Application.java  
  
@SpringBootApplication  
  
@Import(CsvDataLoader.class)  
public class Application {  
  
    @Autowired
```

```

private Auction auction;

public static void main(String[] args) {
    SpringApplication.run(Application.class, args);
}

@Bean
ApplicationRunner configureRepository(UserRepository userRepository)
    return args -> {

        for (User user : auction.getUsers()) {
            userRepository.save(user);
        }

    };
}
}

```

Ⓐ Import the `CsvDataLoader` class and the `Auction` bean it creates.

Ⓑ Autowire the imported `Auction` bean.

Ⓒ Browse all users from the auction, and save them in the repository.

We can access the Spring Data REST application in the browser (<http://localhost:8081/users>) as shown in figure 16.2. We get the information about users and the option to easily navigate between the records. Spring Data REST will expose information about the API to be accessed, providing links to each record.

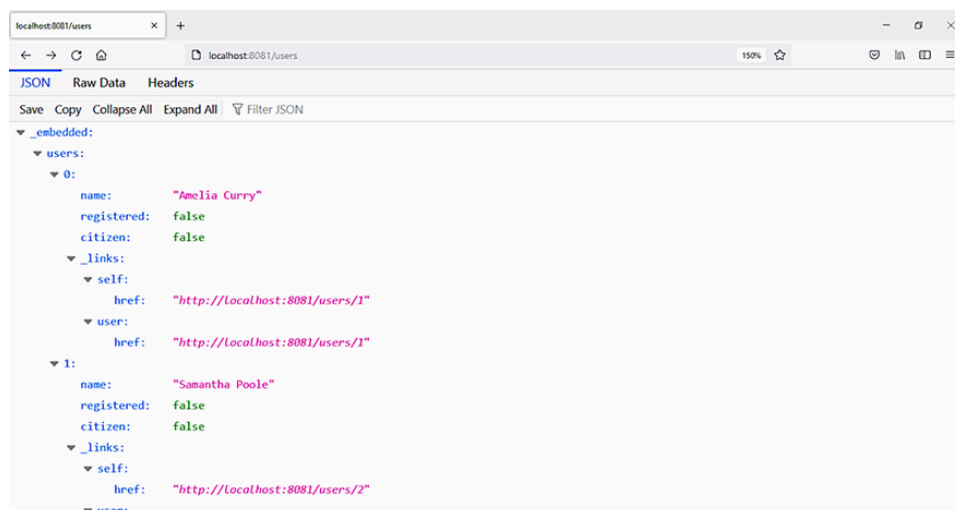


Figure 16.2 Accessing the Spring Data REST application from the browser

We can test this REST API endpoint using a REST client. IntelliJ IDEA Ultimate edition provides such a REST client, but you can use a different client (such as cURL or Postman). We can execute commands like the following (see figure 16.3):


```
GET http://localhost:8081/users/1
```

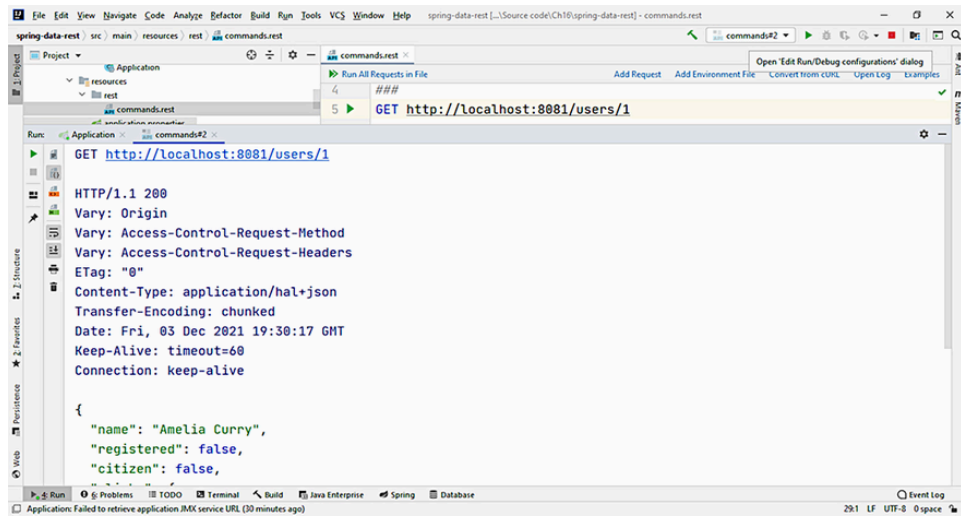


Figure 16.3 The result of executing the `GET http://localhost:8081/users/1` command in the IntelliJ IDEA Ultimate edition REST client

16.3 Using ETags for conditional requests

Any exchange of information across the network requires time. The smaller the information is, the quicker our program will work. But when and how can we reduce the amount of information retrieved from the server and transferred across the network?

Suppose we need to execute a command such as the following one multiple times:

```
GET http://localhost:8081/users/1
```

We'll access the server each time, and the same information will be sent across the network. This is inefficient, and we'd like to limit the amount of data exchanged between the client and server.

We can use ETags to make conditional requests and avoid sending information that has not changed. An ETag is an HTTP response header returned by a web server. It will help us determine if the content at a given URL has been modified, and consequently allow us to make a conditional request.

In the `User` class, there is a field annotated with the `@Version` annotation:

```
@Version
    \1 Long version;
```

This field will also be used as an ETag. When we execute this request to the server,

```
GET http://localhost:8081/users/1
```

the answer will include, on the header, the version of the record (0), as an ETag (see figure 16.4):

```
HTTP/1.1 200
Vary: Origin
Vary: Access-Control-Request-Method
Vary: Access-Control-Request-Headers
ETag: "0"
```

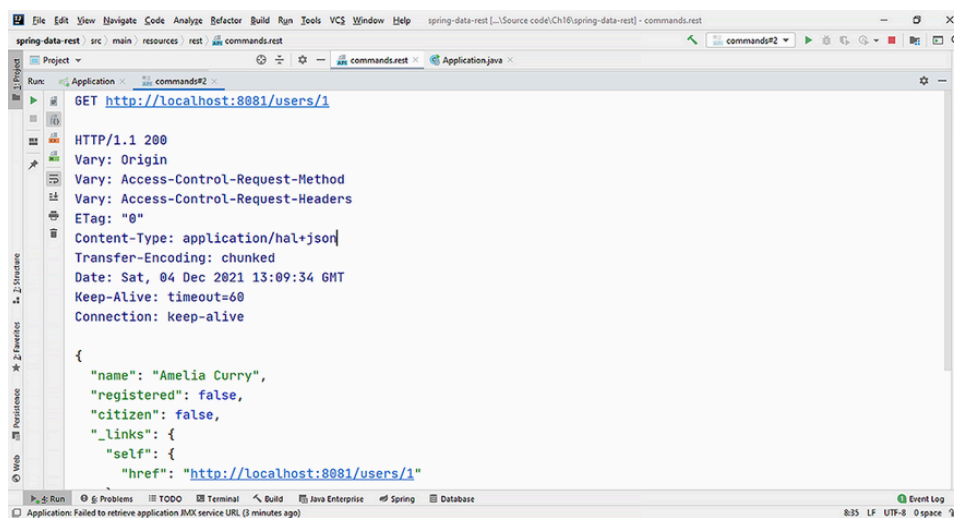


Figure 16.4 The server's answer, including the ETag on the header, representing the entity version

Using this information, we can now execute a conditional request and get the information about the user with ID 1 only if the ETag is different than 0.

```
GET http://localhost:8081/users/1
If-None-Match: "0"
```

The answer from the server will be the 304 (Not Modified) response code, together with an empty body (see figure 16.5):

```
HTTP/1.1 304
Vary: Origin
Vary: Access-Control-Request-Method
Vary: Access-Control-Request-Headers
ETag: "0"
Date: Sat, 04 Dec 2021 13:19:11 GMT
Keep-Alive: timeout=60
Connection: keep-alive
```

<Response body is empty>

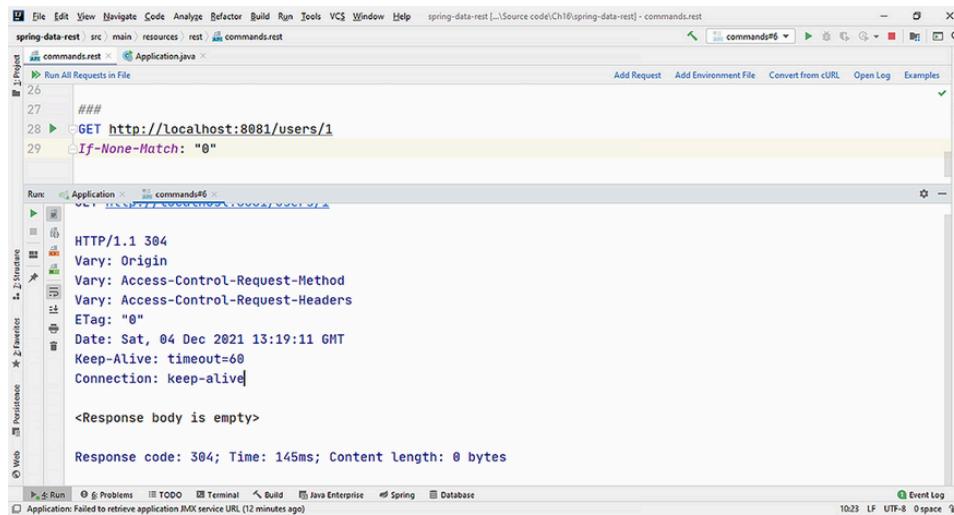


Figure 16.5 The server's answer for a record matching the existing ETag does not have a body.

We can now modify the content from the user with ID 1, executing a `PATCH` command. We use a `PATCH` instead of a `PUT`, as `PATCH` will update only the fields that are included in the request, whereas `PUT` will replace an entire entity with a new one.

```
PATCH http://localhost:8081/users/1
Content-Type: application/json
{
  "name": "Amelia Jones",
  "isRegistered": "true"
}
```

The answer from the server will be 204 (No Content) success response code, and the ETag will be the increased version of the record (1) (see figure 16.6):

```
HTTP/1.1 204
Vary: Origin
Vary: Access-Control-Request-Method
Vary: Access-Control-Request-Headers
ETag: "1"
Date: Sat, 04 Dec 2021 13:25:57 GMT
Keep-Alive: timeout=60
Connection: keep-alive

<Response body is empty>
```

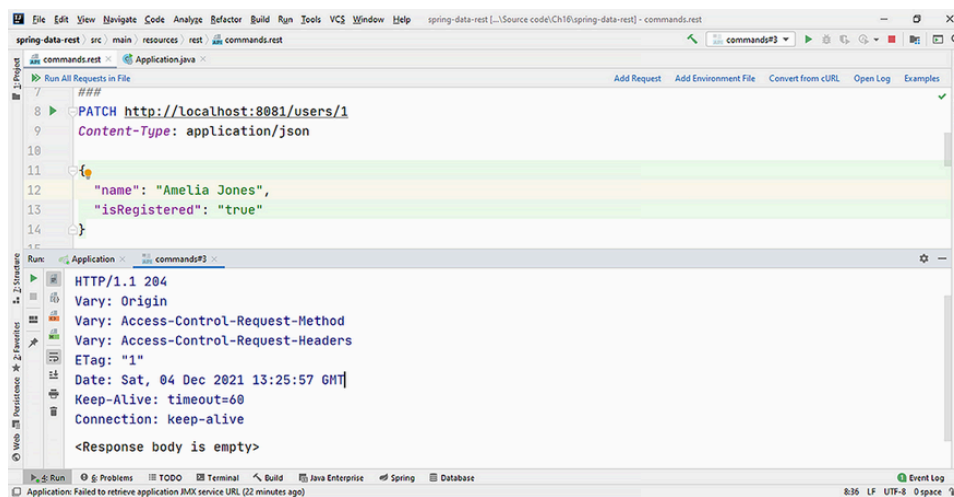


Figure 16.6 The server's answer after patching a user increases the ETag to 1.

We can now re-execute the conditional request, to get the information about user with ID 1 only if the ETag is different from 0:

```
GET http://localhost:8081/users/1
If-None-Match: "0"
```

As the version of the record was changed from 0 to 1, the conditional request will get an answer with the 200 (Success) response code and the full information about the user (see figure 16.7):

```
HTTP/1.1 200
Vary: Origin
Vary: Access-Control-Request-Method
Vary: Access-Control-Request-Headers
ETag: "1"
```

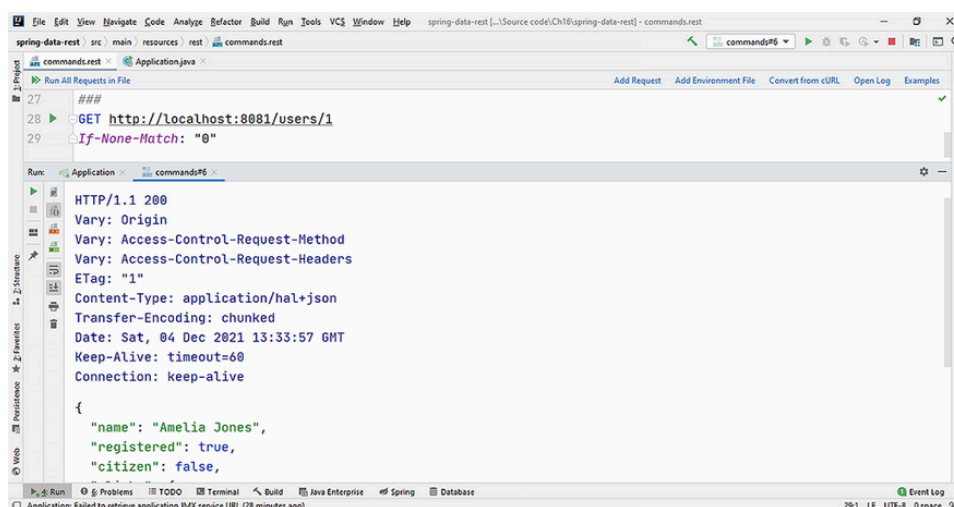


Figure 16.7 The server's answer includes the full information about the user, with the ETag changed from 0 to 1.

16.4 Limiting access to repositories, methods, and fields

Spring Data REST will export all public top-level repository interfaces by default. But real-world use cases will frequently require limiting access to particular methods, fields, or even whole repositories. We can use the `@RepositoryRestResource` annotation to block an *interface* from being exported or to customize access to an endpoint.

For example, if the managed entity is `User`, Spring Data REST will export it to the `/users` path. We can block the export of the entire repository by using the `exported = false` option of the `@RepositoryRestResource` annotation. The repository will look like this:

```
@RepositoryRestResource(path = "users", exported = false)

public interface UserRepository extends JpaRepository<User, Long> {

}
```

Any command executed against this repository will result in an error. For example, executing

```
GET http://localhost:8081/users/1
```

will generate the 404 (Not Found) response code from the server (see figure 16.8):

```
HTTP/1.1 404
Vary: Origin
Vary: Access-Control-Request-Method
Vary: Access-Control-Request-Headers
Content-Type: application/json
```

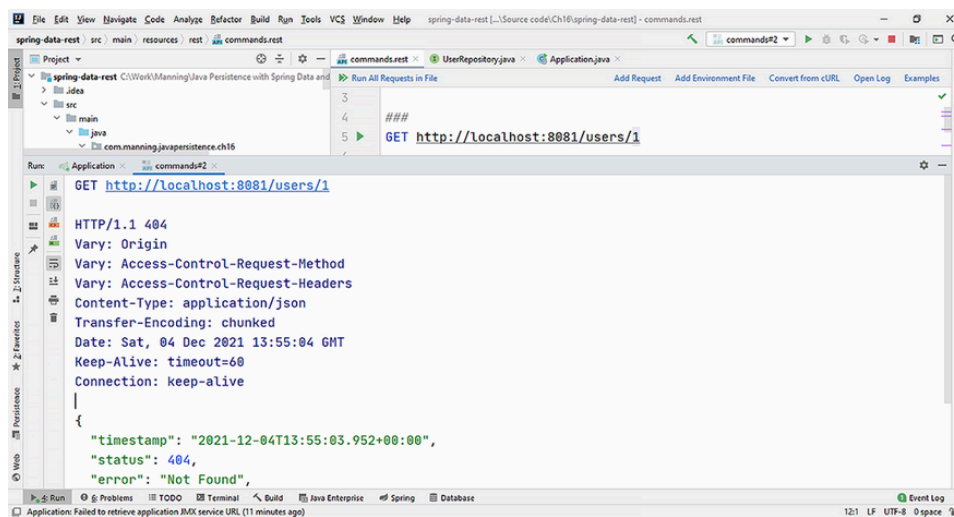


Figure 16.8 Blocking the export of the repository will prevent any interaction from the REST interface.

For convenience, we'll use the `@RepositoryRestResource` annotation with its default options for the `UserRepository` interface.

By default, Spring Data REST will also export all methods from a repository interface, but we can block access to these *methods* with the `@RestResource(exported = false)` annotation. For the `UserRepository` interface, we won't export the deletion methods.

Listing 16.8 The `UserRepository` interface

Path: `Ch16/spring-data-rest/src/main/java/com/manning/javapersistence/c`
 ➔ `/repositories/UserRepository.java`

```
@RepositoryRestResource(path = "users")  Ⓐ
public interface UserRepository extends JpaRepository<User, Long> {

    @Override
    @RestResource(exported = false)  Ⓑ
    void deleteById(Long id);

    @Override
    @RestResource(exported = false)  Ⓑ
    void delete(User entity);
}
```

Ⓐ Use the `@RepositoryRestResource` annotation to export the repository to the `/users` path. This is the default option.

Ⓑ Use the `@RestResource(exported = false)` annotation to not export the delete methods of the repositories.

If we now execute the `DELETE` command,

```
DELETE http://localhost:8081/users/1
```

the server will respond with the 405 (Method Not Allowed) response code, because the delete method was not exported (see figure 16.9). The allowed methods are GET, HEAD, PUT, PATCH, and OPTIONS:

```
HTTP/1.1 405
Vary: Origin
Vary: Access-Control-Request-Method
Vary: Access-Control-Request-Headers
Allow: GET,HEAD,PUT,PATCH,OPTIONS
```

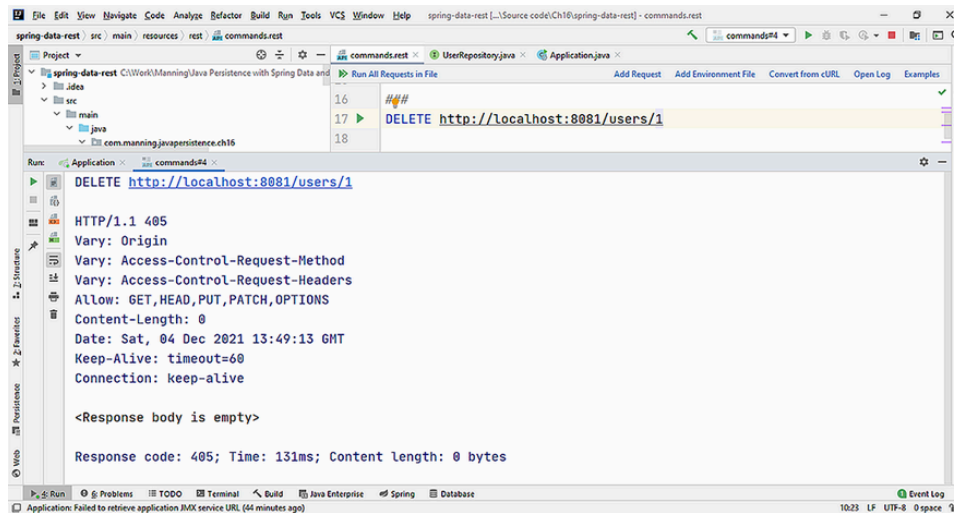


Figure 16.9 The delete method is no longer exported by Spring Data REST and is not allowed by the server.

We can limit access to particular fields and not expose them in the REST interface by using the `@JsonIgnore` annotation. For example, we can use this annotation inside the `User` class, on the `isRegistered` method:

```
@JsonIgnore
public boolean isRegistered() {
    return isRegistered;
}
```

Accessing the repository through the browser will no longer provide the `isRegistered` field information. You can see this in figure 16.10 and compare it to figure 16.2.

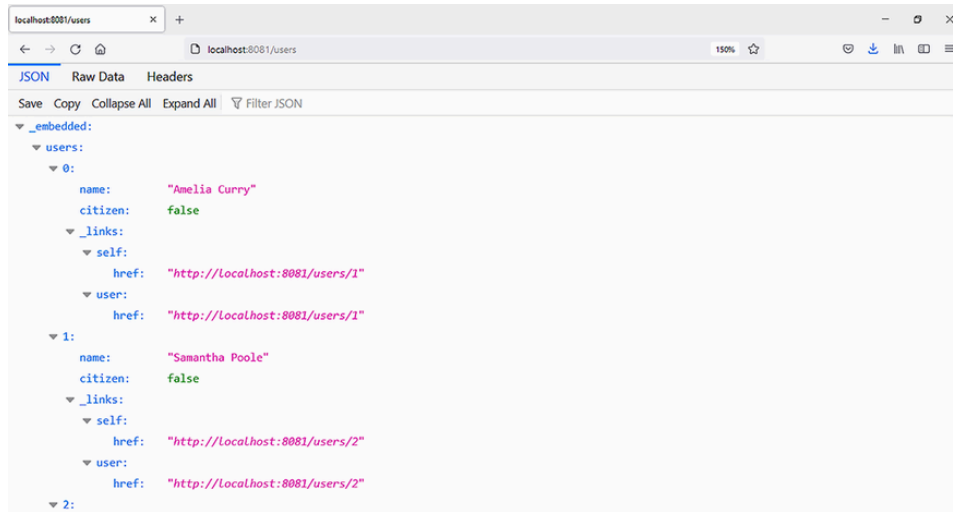


Figure 16.10 The REST client no longer gets the `isRegistered` information.

16.5 Working with REST events

In some situations, we may need to add side effects to the behavior of the application when a particular event occurs. The REST application can emit 10 different types of events when working with an entity. All of them extend the `org.springframework.data.rest.core.event.RepositoryEvent` class and belong to the same `org.springframework.data.rest.core.event` package:

- `BeforeCreateEvent`
- `AfterCreateEvent`
- `BeforeSaveEvent`
- `AfterSaveEvent`
- `BeforeLinkSaveEvent`
- `AfterLinkSaveEvent`
- `BeforeDeleteEvent`
- `AfterDeleteEvent`
- `BeforeLinkDelete`
- `AfterLinkDelete`

These events can be treated in two ways:

- Write an annotated handler
- Write an `ApplicationListener`

Let's look at these two options.

16.5.1 Writing an AnnotatedHandler

To add side effects by writing an `AnnotatedHandler`, we can create a POJO class with the `@RepositoryEventHandler` annotation on it. This annotation tells the `BeanPostProcessor` managed by Spring that this class must be inspected for handler methods. The `BeanPostProcessor`

will browse the methods of the class carrying this annotation and detect annotations that correspond to different events.

The event handler bean must be under the control of the container. We can annotate the class as a `@Service` (which is a `@Component` stereotype), so it will be considered by `@ComponentScan` or `@SpringBootApplication`.

The entity whose events we are following is provided by the type of the first parameter of the annotated methods. In the following examples, the methods of the handler will have a `User` entity as a parameter.

The association between the annotated methods and the events is summarized in table 16.1.

Table 16.1 `AnnotatedHandler` annotations and corresponding events

Annotations	Events
<code>@HandleBeforeCreate</code> <code>@HandleAfterCreate</code>	POST event
<code>@HandleBeforeSave</code> <code>@HandleAfterSave</code>	PUT and PATCH events
<code>@HandleBeforeDelete</code> <code>@HandleAfterDelete</code>	DELETE events
<code>@HandleBeforeLinkSave</code> <code>@HandleAfterLinkSave</code>	A linked object is saved to the repository
<code>@HandleBeforeLinkDelete</code> <code>@HandleAfterLinkDelete</code>	A linked object is deleted from the repository

The `UserRepositoryEventHandler` class, a POJO class with the `@RepositoryEventHandler` annotation on it, is shown in the following listing.

Listing 16.9 The `UserRepositoryEventHandler` class

```
Path: Ch16/spring-data-rest-events/src/main/java/com/manning
➡ /javapersistence/ch16/events/UserRepositoryEventHandler.java
```

```
@RepositoryEventHandler  Ⓐ
@Service                Ⓑ
public class UserRepositoryEventHandler {
    @HandleBeforeCreate  Ⓒ
    public void handleUserBeforeCreate(User user) {  Ⓓ
```

```

        //manage the event
    }

    //other methods
}

```

Ⓐ Annotate the class with the `@RepositoryEventHandler` annotation to tell the Spring `BeanPostProcessor` to inspect it for handler methods.

Ⓑ Annotate the class with the `@Service` annotation to bring it under the control of the container.

Ⓒ Annotate the method with `@HandleBeforeCreate` to associate it with the `POST` event.

Ⓓ The method has an entity `User` as a first parameter, indicating the type whose events we are following.

16.5.2 Writing an `ApplicationListener`

To add side effects by writing an `ApplicationListener`, we'll extend the `AbstractRepositoryEventListener` abstract class. This class is generified by the type of entity on which the events happen. It will listen for the events and call the corresponding methods. We'll annotate the custom listener as a `@Service` (which is a `@Component` stereotype), so it will be considered by `@ComponentScan` or `@SpringBootApplication`.

The `AbstractRepositoryEventListener` abstract class already contains a series of empty protected methods to address events. We'll need to override and make public only the ones we are interested in.

The association between the methods and events is summarized in table 16.2.

Table 16.2 ApplicationListener methods and corresponding events

Methods	Events
onBeforeCreate onAfterCreate	POST event
onBeforeSave onAfterSave	PUT and PATCH events
onBeforeDelete onAfterDelete	DELETE events
onBeforeLinkSave onAfterLinkSave	A linked object is saved to the repository
onBeforeLinkDelete onAfterLinkDelete	A linked object is deleted from the repository

The `RepositoryEventListener` class, extending the `AbstractRepositoryEventListener` abstract class, contains the methods that react to the events. It's shown in the following listing.

Listing 16.10 The `RepositoryEventListener` class

```
Path: Ch16/spring-data-rest-events/src/main/java/com/manning
➡ /javapersistence/ch16/events/RepositoryEventListener.java
```

```

@Service                                     ❶
public class RepositoryEventListener extends  ❷
    AbstractRepositoryEventListener<User> {  ❸

    @Override                               ❹
    public void onBeforeCreate(User user) {  ❺
        //manage the event
    }
    //other methods
}
```

❶ Annotate the class with the `@Service` annotation to bring it under the control of the container.

❷ Extend the `AbstractRepositoryEventListener`, generified by the `User` entity—the entity on which the events happen.

❸ The method has a `User` entity as the first parameter, indicating the type whose events we are following.

Now we can run the application and execute a REST command as follows:

```
POST http://localhost:8081/users
Content-Type: application/json
{
  "name": "John Smith"
}
```

The handler and the listener will react to the events and generate additional behavior as a side effect, as shown in figure 16.11.

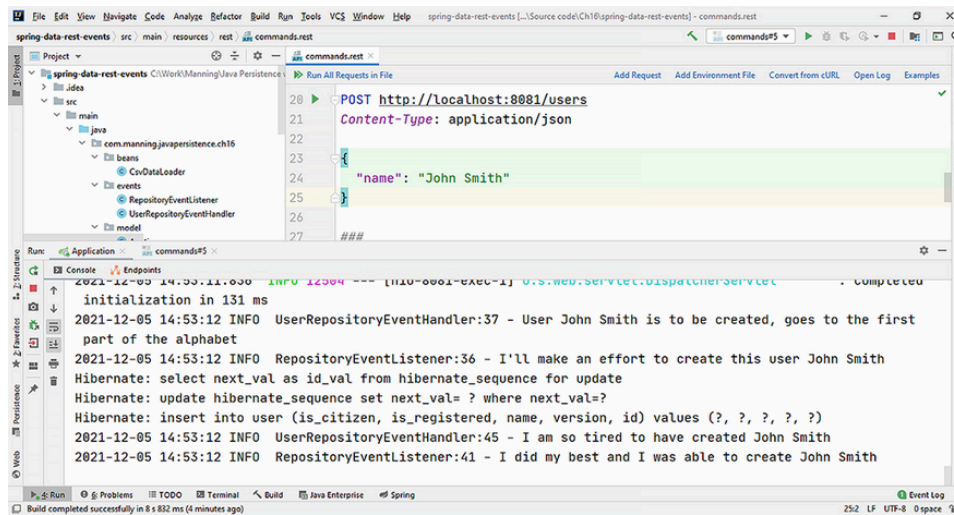


Figure 16.11 Additional behavior (side effects) from the handler and listener reacting to a REST event

The two approaches of addressing events (with handlers and with listeners) provide similar behavior and they treat the same types of events. All other things being equal, handlers provide the advantage of working only at a declarative level (annotations on classes and methods), while listeners require us to extend an existing abstract class, so they hang in an existing hierarchy, which means less freedom of hierarchy design.

16.6 Using projections and excerpts

Spring Data REST provides a default view of the domain model you are working with, but real-world use cases may require it to be changed or adapted to particular needs. You can do this using projections and excerpts, providing specific views of the information that is exported.

We'll add the new `Address` class to the project. It will contain a few fields, and we'd like to display the information inside it with the `toString` method.

Listing 16.11 The `Address` class

```
Path: Ch16/spring-data-rest-projections/src/main/java/com/manning
➡ /javapersistence/ch16/model/Address.java
```

```
@Entity
public class Address {

    @GeneratedValue
    @Id
    private Long id;
    private String street, zipCode, city, state;

    //constructors and methods

    public String toString() {
        return String.format("%s, %s %s, %s", street, zipCode, city, stat
    }
}
```

There is a one-to-one relationship between `User` and `Address`, as we introduced a new field in the `User` entity:

```
@OneToOne(cascade = CascadeType.ALL, orphanRemoval = true)
private Address address;
```

The `CascadeType.ALL` option will cause the persistence operations to be cascaded to the related entities. The `orphanRemoval=true` argument specifies that we want to permanently remove an `Address` when it is no longer referenced by the `User`. You can revisit chapter 8 for more details about these options.

If we access the `http://localhost:8081/users/1` URL, we'll get the default view of the user with ID 1, displaying all its fields and the fields from the address, as shown in figure 16.12.

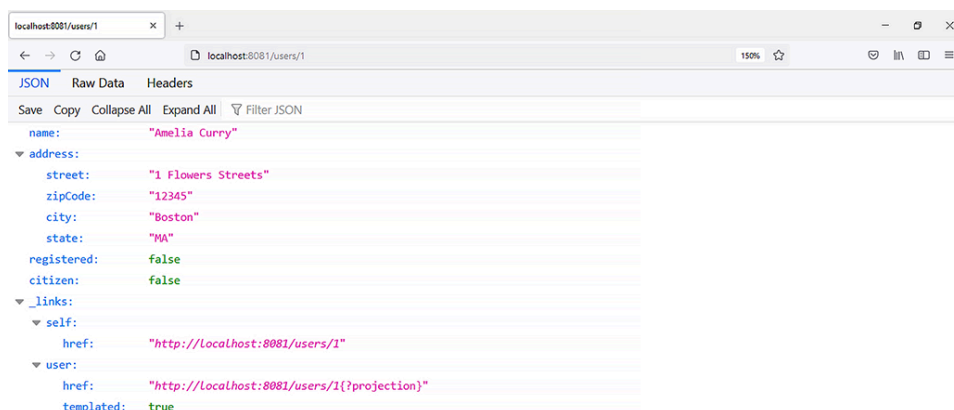


Figure 16.12 The default view of a user with an address

We'll now add the new `UserProjection` interface to the project (listing 16.12). With the help of the `@Projection` annotation, we can create the

summary projection on the `User` entity, which will export only the name of the user and the address according to how it is displayed by the `toString` method. We'll do this using Spring Expression Language (SpEL).

Listing 16.12 The `UserProjection` interface

```
Path: Ch16/spring-data-rest-projections/src/main/java/com/manning
➡ /javapersistence/ch16/model/UserProjection.java
```

```
@Projection(name = "summary", types = User.class)    ❶
public interface UserProjection {

    String getName();                                ❷

    @Value("#{target.address.toString()}")            ❸
    String getAddress();                              ❹
}
```

❶ The projection is named `summary`, and it applies to the `User` entities.

❷ As the field is called `name`, we need to write a `getName` method to export it, following the getter names convention.

❸ Export the `address` according to how it is displayed by the `toString` method. We use the `@Value` annotation, containing a SpEL expression. We also need to follow the getter names convention, so the method is called `getAddress`.

If we access the `http://localhost:8081/users/1?projection=summary` URL (with the projection name included as a parameter), we'll get a view of the user with ID 1, displaying the `name` field and the address provided by the `toString` method. This is shown in figure 16.13.

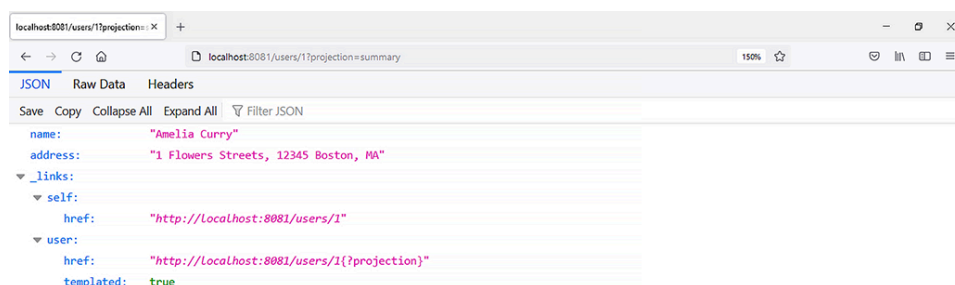


Figure 16.13 The view of a user with an address provided by the `summary` projection

We may want to apply the default view of a projection at the level of a whole collection. In this case, we'll have to go to the already defined repository and use the `excerptProjection =`

UserProjection.class option of the @RepositoryRestResource annotation, as shown in listing 16.13.

Listing 16.13 The modified UserRepository interface

```
Path: Ch16/spring-data-rest-projections/src/main/java/com/manning
➡ /javapersistence/ch16/repositories/UserRepository.java

@RepositoryRestResource(path = "users",
                        excerptProjection = UserProjection.class)
public interface UserRepository extends JpaRepository<User, Long> {
}
```

If we access the `http://localhost:8081/users/` URL, we'll get a view of all users, displayed according to the projection definition, as shown in figure 16.14.

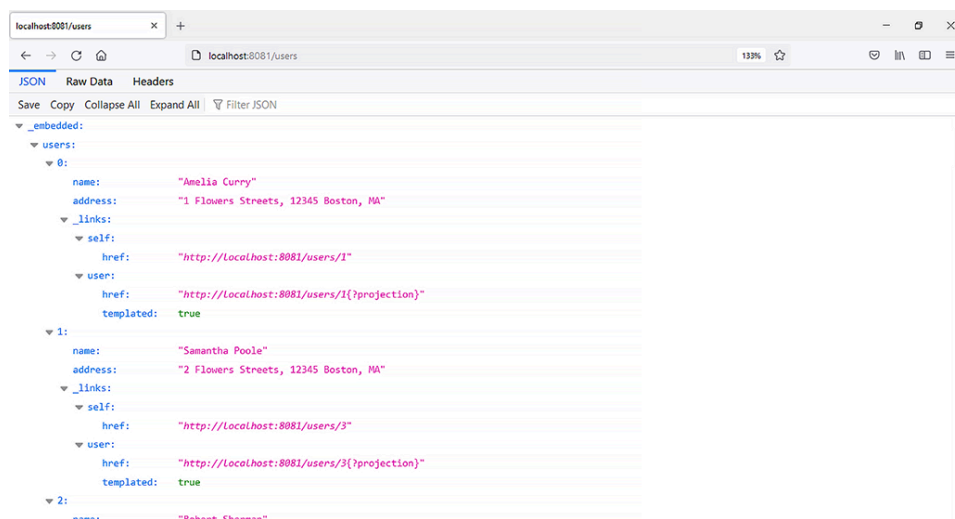


Figure 16.14 The view of the whole `users` collection, displayed according to the `summary` projection

Summary

- Using Spring Boot, you can create and configure a Spring Data REST project to provide an interface to interact with the database and to manage and persist information.
- You can use ETags to make efficient requests that get data from the server, avoiding transferring information that the client already has.
- You can limit access to repositories, methods, and fields, and export only the information and actions that you would like to allow.
- You can work with REST events and manage them through handlers and listeners. They can work through meta-information or by extending an existing class.
- You can use projections and excerpts to provide customized views of the information exported by the repository, according to the needs of different users.

